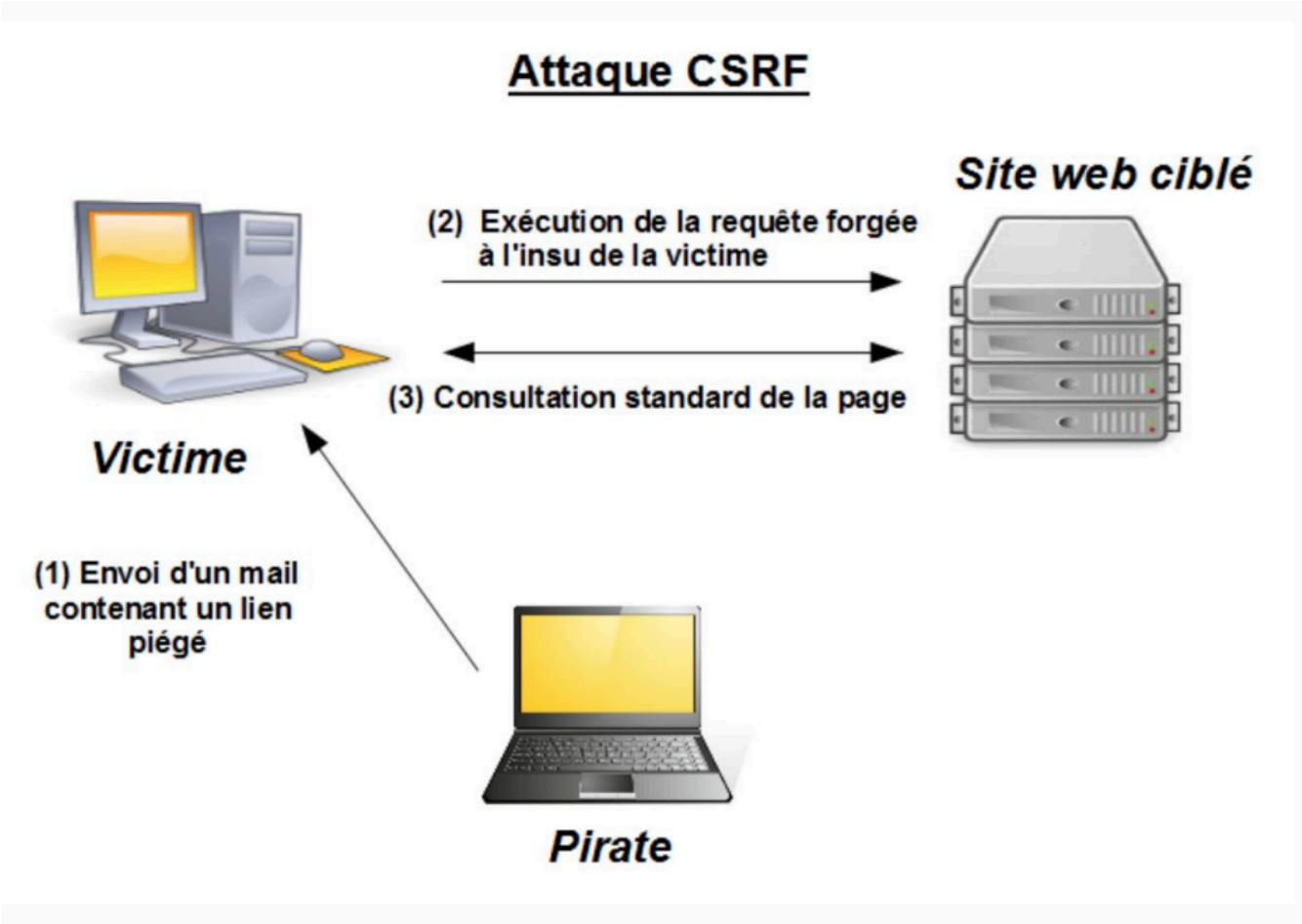
Attaque CRSF

Ecrit par Valentin CEMON

Les attaques de type CSRF (Cross Site Request) exploitent la confiance d'un utilisateur authentifié pour effectuer des actions non autorisées sur le web auquel l'utilisateur est connecté.

Le fonctionneement d'une attaque CSRF :

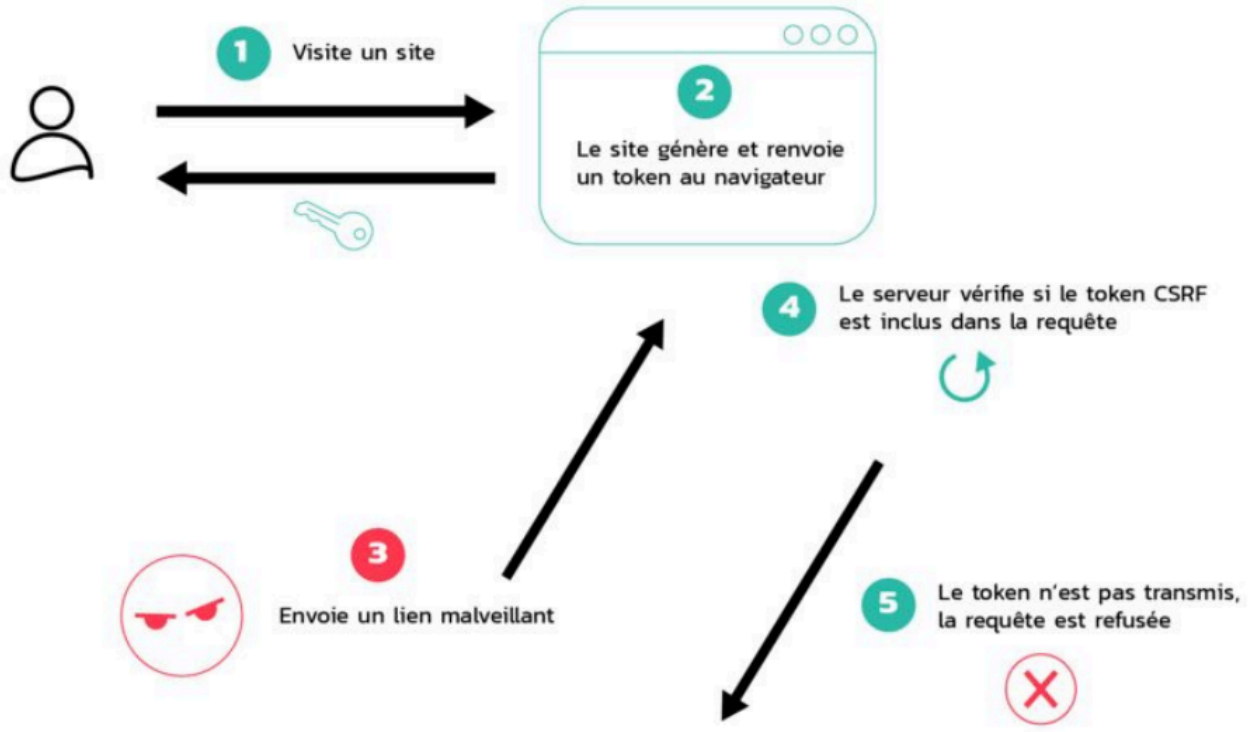
1. L'utilisateur authentifié se connecte à un site web (par exemple, une application bancaire) et reçois une cookie de session.
2. Pendant la session active, l'utilisateur visite un autre site web, généralement compromis, qui contient un lien ou une forme malveillante. Ce lien ou cette forme effectue des actions sur le site cible sans le consentement de l'utilisateur.
3. Lorsque l'utilisateur clique sur le lien ou soumet le formulaire sur le site compromis, les actions malveillantes sont effectuées sur le site cible en utilisant les informations d'authentificaiton (cookies) de l'utilisateur, car le navigateur envoie automatiquement les cookies avec la requête.



Pour se protéger contre les attaques CSRF, une mesure de sécurité courante consiste en l’implémentation de jetons CSRF.

Les jetons CSRF sont des valeurs uniques générées aléatoirement côté serveur et envoyées au client. Ces valeurs étant uniques pour chaque requête, elles permettent de renforcer la sécurité d'une application en rendant difficile (voire impossible) pour un attaquant de les deviner et donc d'exploiter une vulnérabilité CSRF.

Par ailleurs, certains frameworks, tels que Django (Python) et Laravel (PHP), intègrent des middlewares activés par défaut assurant une protection contre les attaques CSRF.



Utiliser l'attribut SameSite sur les cookies pour contrer les attaques CSRF

SameSite est un attribut qu'une application peut insérer sur les cookies. En effet, lorsque cet attribut est placé sur les cookies de sessions, ceux-ci ne seront pas envoyés au serveur si la requête ne provient pas du domaine de l'application.

L'application recevant la requête ne sera pas capable de relier la requête à un quelconque utilisateur et par conséquent ne la traitera pas. Cela permet donc en théorie de se protéger totalement contre les attaques CSRF.

Regardons de plus près le fonctionnement de l'attribut SameSite :

L'application demande au navigateur de l'utilisateur de créer des cookies via un en-tête Set-Cookie. Par exemple, si nous avons affaire à une application PHP/Laravel, il est très probable que nous ayons une valeur d'en-tête similaire à celle-ci :

```
Set-Cookie: PHPSESSID=VxTGA2T1T0kE
```

L'en-tête Set-Cookie autorise plusieurs indicateurs afin d'améliorer la sécurité des cookies de l'utilisateur : SameSite, Expires, Domain, etc.

SameSite peut être utilisé avec différentes valeurs : Strict et Lax. Lorsque l’attribut SameSite est défini sur « Strict » (SameSite=Strict), cela signifie que le navigateur de l’utilisateur n’enverra jamais de cookies avec les requêtes inter-sites. Le cookie n’est transmis que pour les demandes provenant du même domaine et sous-domaines.

```
Set-Cookie: PHPSESSID=VxTGA2T1T0kE; Max-Age=43200; Secure; HttpOnly; SameSite=Strict
```

Si l’attribut « Lax » est défini (SameSite=Lax), le navigateur du client recevra l’instruction d’envoyer des cookies uniquement sur les requêtes GET qui peuvent provoquer une navigation entre domaines de premier niveau. Cette configuration garantit que les clients auront toujours accès aux ressources de votre site web si l’interaction inter-site est voulue.

```
Set-Cookie: PHPSESSID=VxTGA2T1T0kE; Max-Age=43200; Secure; HttpOnly; SameSite=Lax
```